Table of Contents

PREAMBLE

DO NOT REMOVE THE LINE BELOW

```
clear; close all;
```

==========

==========

1. COMMENTING

==========

==========

MAKE SURE FILE BELOW IS IN SAME DIRECTORY AS THIS FILE

```
type('eel3135_lab08_comment.m')
```

```
%% QUESTION #1 COMMENTING
clear
close all
clc
```

```
%% DEFINE FILTER AND INPUT
w = -pi:pi/8000:pi-pi/8000;
N = 100;
n = 0:(N-1);

% FILTER 1
a1 = [1 0 0 0 0 0 0 -0.9];
b1 = 1;
% <-- Answer: Is this an IIR filter or a FIR filter? Why?
% This is an IIR filter because the denominator is not equal to one, and
there are poles not at the origin.
% This means that the system has feedback and is an IIR.
% <-- Answer: How many poles does this system have? How many zeros?
% 7 poles and 7 zeros

% FILTER 2
a2 = [1 -0.9];
b2 = 1;
% <-- Answer: Is this an IIR filter or a FIR filter? Why?
% This is an IIR filter because the denominator is not equal to one, and
there is a pole not at the origin.
% This means that the system has feedback and is an IIR.
% <-- Answer: How many poles does this system have? How many zeros?
% 1 pole and 1 zero

% INPUT
x = zeros(N,1);
x(1) = 1;

%% DEFINE AND PLOT OUTPUT

% OUTPUT 1
y1 = filter(b1,a1,x);
y2 = filter(b2,a2,x);

% OUTPUT 2
H1 = DTFT(y1,w);
H2 = DTFT(y2,w);

% OUTPUT 3: CASCADE FILTERS
y3 = filter(b2, a2, filter(b1, a1, x));
H3 = DTFT(y3,w);
% <-- Express H3(z) as a function of H1(z) and H2(z)
% H3(z) = H1(z)H2(z), since cascading two LTI systems multiplies
% their transfer functions.


% PLOT THE FREQUENCY REPONSE IMPULSE RESPONSE AND DTFT
figure(1)
subplot(3,1,1)
stem(n,y1)
xlabel('Time (Samples)')
ylabel('h[n]')
```

```
subplot(3,1,2)
plot(w,abs(H1))
title('Magnitude Response of h')
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
subplot(3,1,3)
pzplot(b1,a1)
axis equal

% PLOT THE FREQUENCY REPONSE IMPULSE RESPONSE AND DTFT
figure(2)
subplot(3,1,1)
stem(n,y2)
xlabel('Time (Samples)')
ylabel('h[n]')
subplot(3,1,2)
plot(w,abs(H2))
title('Magnitude Response of h')
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
subplot(3,1,3)
pzplot(b2,a2)
axis equal

% PLOT THE FREQUENCY REPONSE IMPULSE RESPONSE AND DTFT
figure(3)
subplot(2,1,1)
stem(n,y3)
xlabel('Time (Samples)')
ylabel('h[n]')
subplot(2,1,2)
plot(w,abs(H3))
title('Magnitude Response of h')
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
```

```
%% ALL FUNCTIONS SUPPORTING THIS CODE %%
% ======================================================================
% NOTE: YOU DO NOT NEED TO ADD COMMENTS IN THE CODE BELOW. WE JUST
% NEEDED POLE-ZERO PLOTTING CODE AND THUS WROTE IT.
% ======================================================================
function pzplot(b,a)
% PZPLOT(B,A)  plots the pole-zero plot for the filter described by
% vectors A and B.  The filter is a "Direct Form II Transposed"
% implementation of the standard difference equation:
%
%    a(1)*y(n) = b(1)*x(n) + b(2)*x(n-1) + ... + b(nb+1)*x(n-nb)
%                          - a(2)*y(n-1) - ... - a(na+1)*y(n-na)
%
```

```
    % MODIFY THE POLYNOMIALS TO FIND THE ROOTS
    b  = b(1:find(b,1,'last'));
    a  = a(1:find(a,1,'last'));
    b1 = zeros(max(length(a),length(b)),1); % Need to add zeros to get the
right roots
    a1 = zeros(max(length(a),length(b)),1); % Need to add zeros to get the
right roots
    b1(1:length(b)) = b;    % New a with all values
    a1(1:length(a)) = a;    % New a with all values

    % FIND THE ROOTS OF EACH POLYNOMIAL AND PLOT THE LOCATIONS OF THE ROOTS
    h1 = plot(real(roots(a1)), imag(roots(a1)));
    hold on;
    h2 = plot(real(roots(b1)), imag(roots(b1)));
    hold off;

    % DRAW THE UNIT CIRCLE
    circle(0,0,1)

    % MAKE THE POLES AND ZEROS X's AND O's
    set(h1, 'LineStyle', 'none', 'Marker', 'x', 'MarkerFaceColor','none',
'linewidth', 1.5, 'markersize', 8);
    set(h2, 'LineStyle', 'none', 'Marker', 'o', 'MarkerFaceColor','none',
'linewidth', 1.5, 'markersize', 8);
    axis equal;

    % DRAW VERTICAL AND HORIZONTAL LINES
    xminmax = xlim();
    yminmax = ylim();
    line([xminmax(1) xminmax(2)],[0 0], 'linestyle', ':', 'linewidth', 0.5,
'color', [1 1 1]*.1)
    line([0 0],[yminmax(1) yminmax(2)], 'linestyle', ':', 'linewidth', 0.5,
'color', [1 1 1]*.1)

    % ADD LABELS AND TITLE
    xlabel('Real Part')
    ylabel('Imaginary Part')
    title('Pole-Zero Plot')

end

function circle(x,y,r)
% CIRCLE(X,Y,R)  draws a circle with horizontal center X, vertical center
% Y, and radius R.
%

    % ANGLES TO DRAW
    ang=0:0.01:2*pi;

    % DEFINE LOCATIONS OF CIRCLE
    xp=r*cos(ang);
    yp=r*sin(ang);

    % PLOT CIRCLE
```

```
    hold on;
    plot(x+xp,y+yp, ':', 'linewidth', 0.5, 'color', [1 1 1]*.1);
    hold off;

end

function H = DTFT(x,w)
% DTFT(X,W)  compute the Discrete-time Fourier Transform of signal X
% acroess frequencies defined by W.

    H = zeros(length(w),1);
    for nn = 1:length(x)
        H = H + x(nn).*exp(-1j*w.'*(nn-1));
    end

end
```

==========

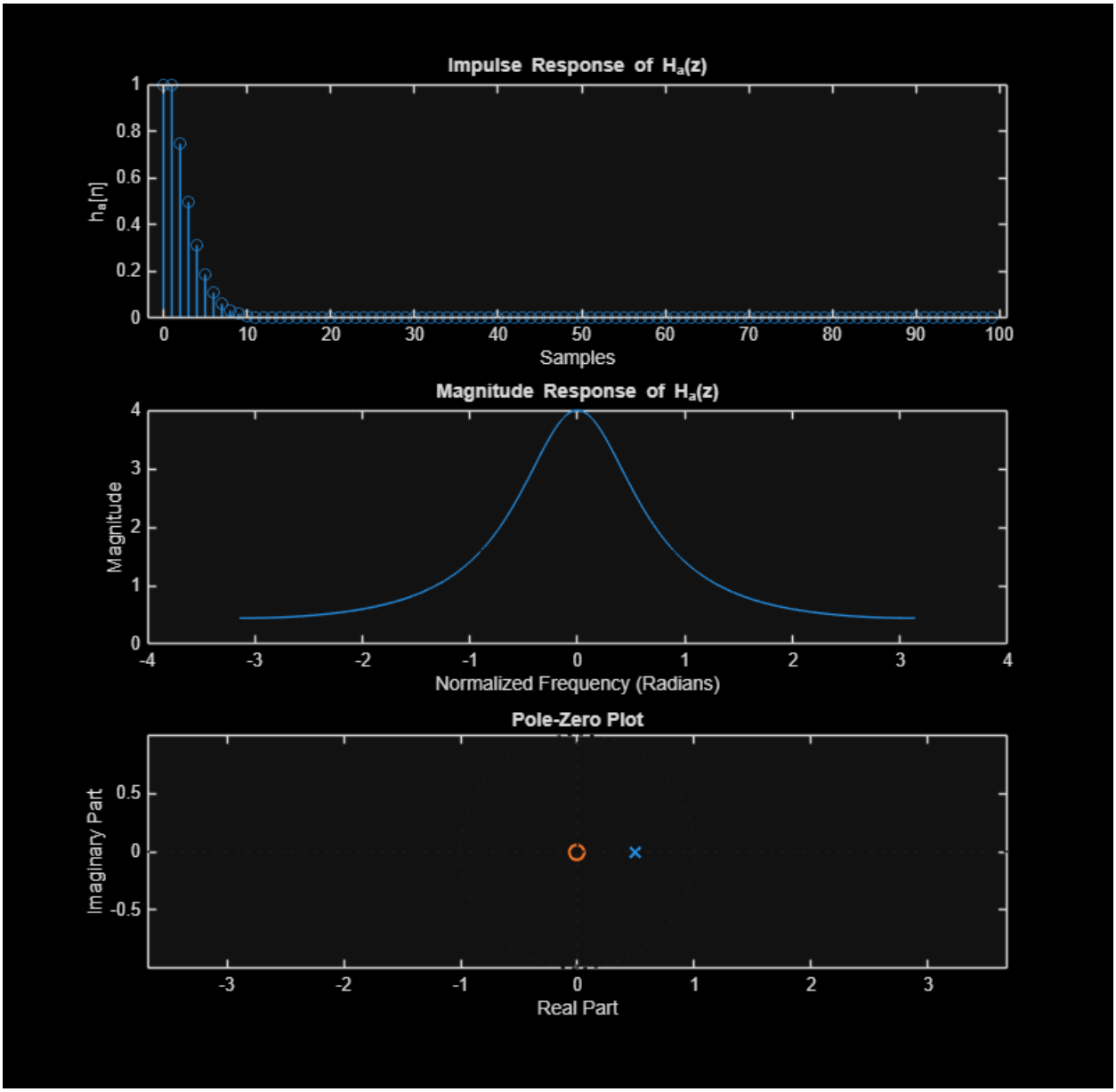
2. IIR Filters in Z-Domain

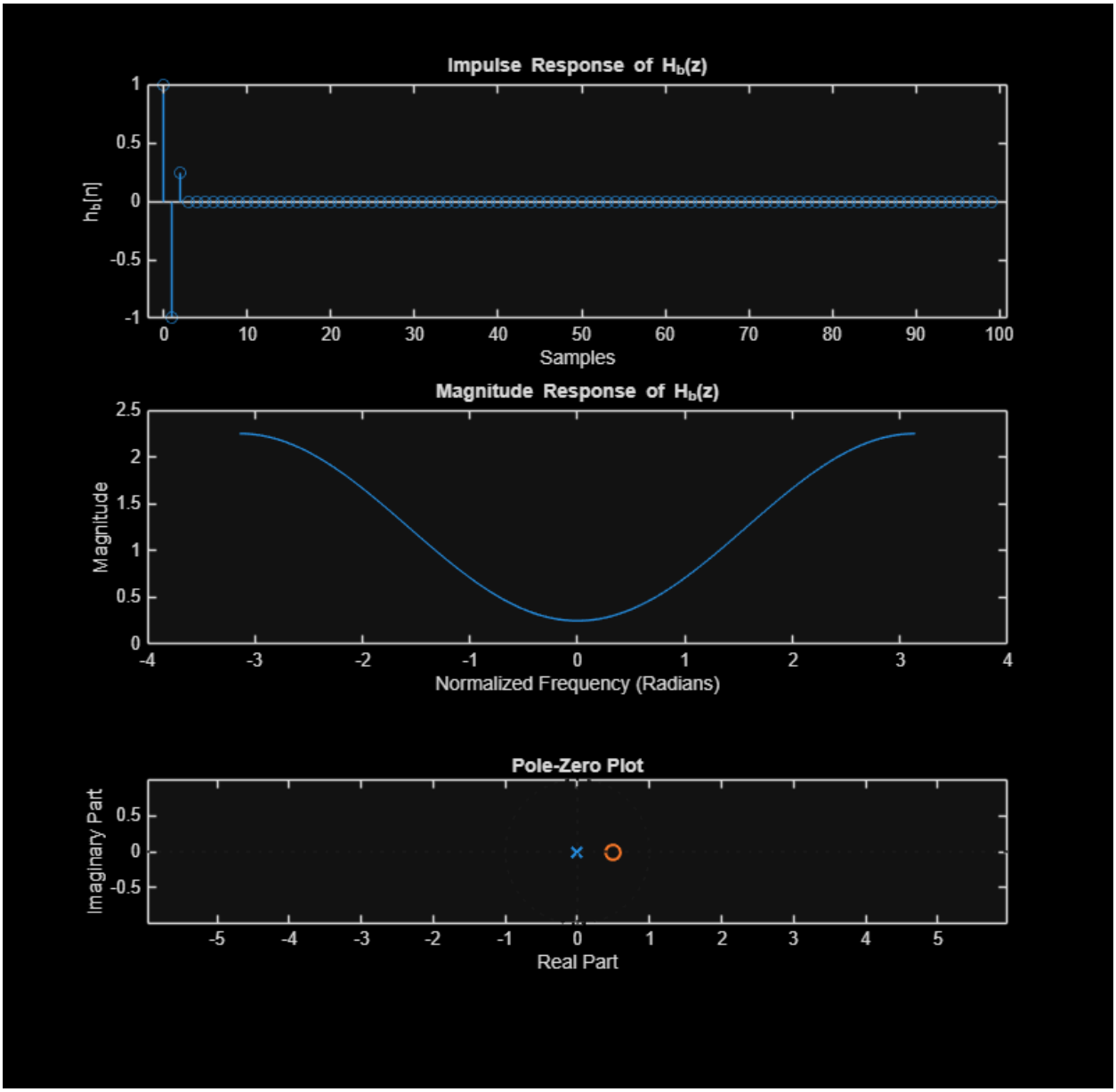
==========

---------- 2(a) ----------

```
wa = 0;
wb = 0;
alpha = 0.5;
beta = 0.5;

N = 100;
n = 0:N-1;
x = zeros(N,1);
x(1) = 1;
w = -pi:pi/8000:pi-pi/8000;

% Ha(z) = 1 / [(1 - alpha*exp(jwa)z^-1)(1 - alpha*exp(-jwa)z^-1)]
a_ha = conv([1 -alpha*exp(1j*wa)], [1 -alpha*exp(-1j*wa)]);
b_ha = 1;

% Hb(z) = [(1 - beta*exp(jwb)z^-1)(1 - beta*exp(-jwb)z^-1)]
b_hb = conv([1 -beta*exp(1j*wb)], [1 -beta*exp(-1j*wb)]);
a_hb = 1;

% impulse responses
h_ha = filter(b_ha, a_ha, x);
h_hb = filter(b_hb, a_hb, x);
```

```
% DTFTs
H_ha = DTFT(h_ha, w);
H_hb = DTFT(h_hb, w);

% Ha plots
figure
subplot(3,1,1)
stem(n, h_ha)
xlabel('Samples')
ylabel('h_a[n]')
title('Impulse Response of H_a(z)')

subplot(3,1,2)
plot(w, abs(H_ha))
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
title('Magnitude Response of H_a(z)')

subplot(3,1,3)
pzplot(b_ha, a_ha)
axis equal

% Hb plots
figure
subplot(3,1,1)
stem(n, h_hb)
xlabel('Samples')
ylabel('h_b[n]')
title('Impulse Response of H_b(z)')

subplot(3,1,2)
plot(w, abs(H_hb))
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
title('Magnitude Response of H_b(z)')

subplot(3,1,3)
pzplot(b_hb, a_hb)
axis equal

% % snapnow;
% ----------------------------------------------------------------
% 2(b)
% ----------------------------------------------------------------
```


Impulse Response of $H_a(z)$
$h_a[n]$
Samples
Magnitude Response of $H_a(z)$
Magnitude
Normalized Frequency (Radians)
Pole-Zero Plot
Imaginary Part
Real Part

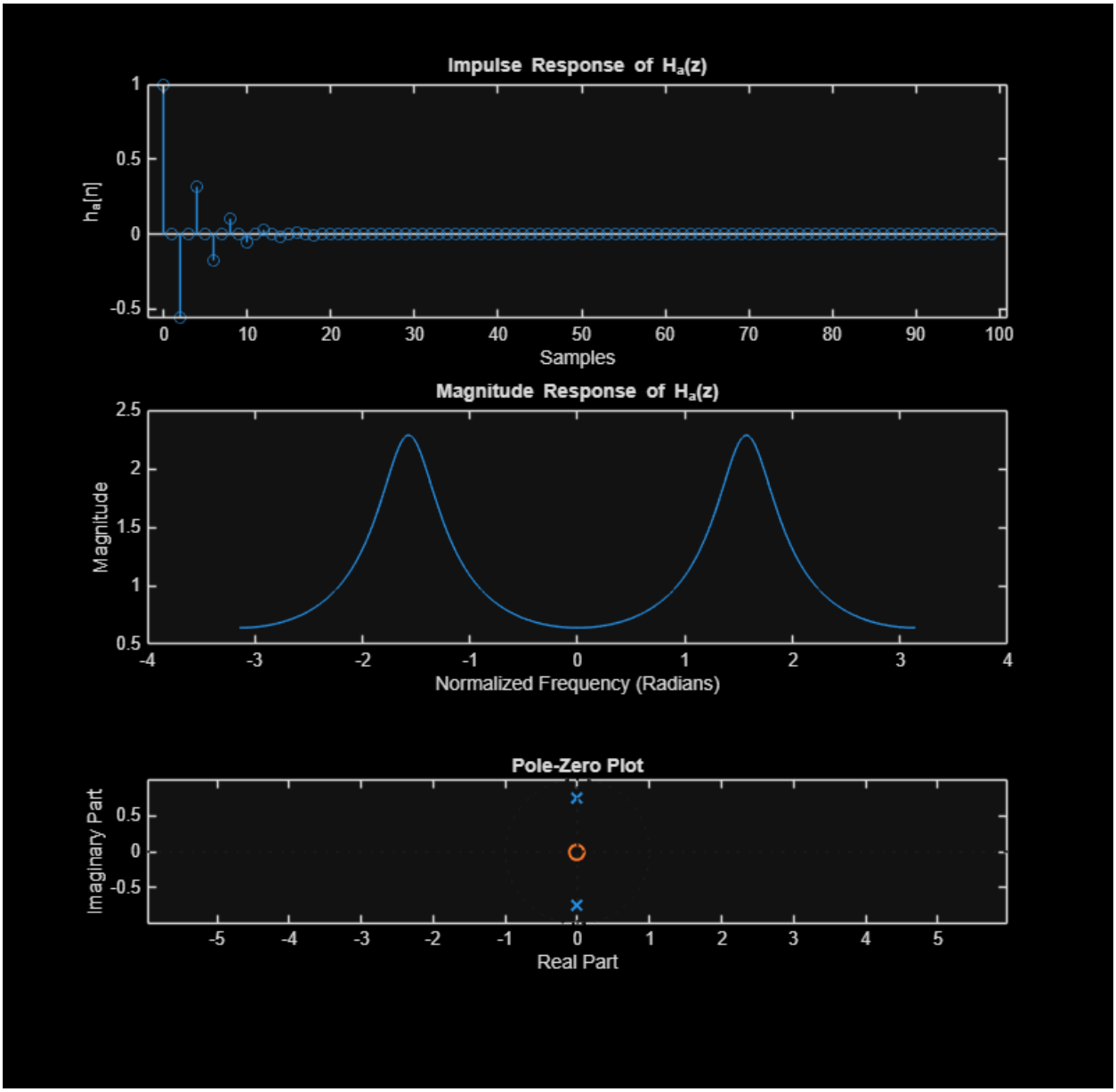


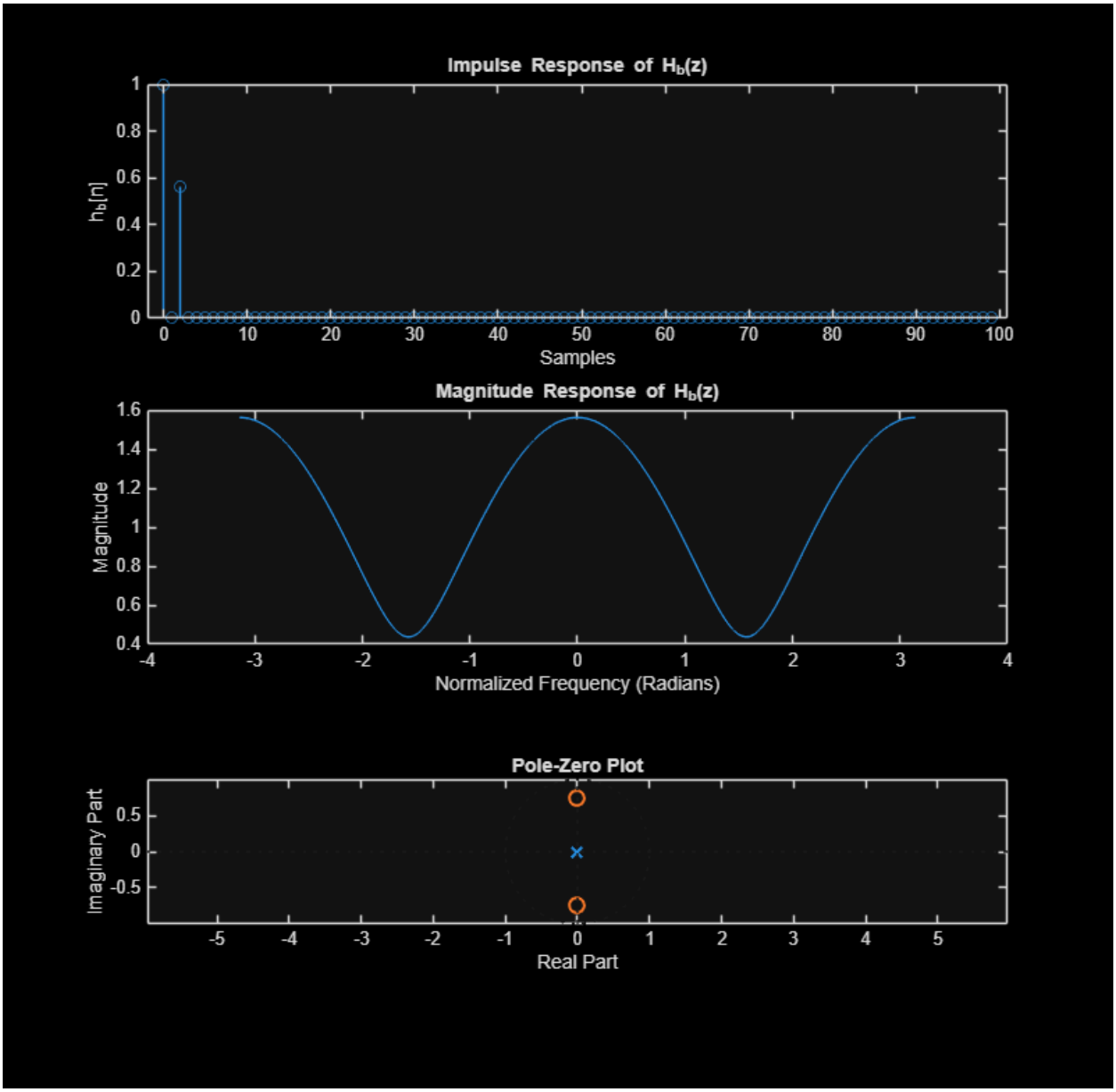
QUESTION 2(b)

```
wa = pi/2;
wb = pi/2;
alpha = 0.75;
beta = 0.75;

N = 100;
n = 0:N-1;
x = zeros(N,1);
x(1) = 1;
w = -pi:pi/8000:pi-pi/8000;

% Ha(z)
```

```
a_ha = conv([1 -alpha*exp(1j*wa)], [1 -alpha*exp(-1j*wa)]);
b_ha = 1;

% Hb(z)
b_hb = conv([1 -beta*exp(1j*wb)], [1 -beta*exp(-1j*wb)]);
a_hb = 1;

% remove tiny imaginary parts from roundoff
a_ha = real(a_ha);
b_hb = real(b_hb);

% impulse responses
h_ha = filter(b_ha, a_ha, x);
h_hb = filter(b_hb, a_hb, x);

% DTFTs
H_ha = DTFT(h_ha, w);
H_hb = DTFT(h_hb, w);

% Ha plots
figure
subplot(3,1,1)
stem(n,h_ha)
xlabel('Samples')
ylabel('h_a[n]')
title('Impulse Response of H_a(z)')

subplot(3,1,2)
plot(w,abs(H_ha))
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
title('Magnitude Response of H_a(z)')

subplot(3,1,3)
pzplot(b_ha,a_ha)
axis equal

% Hb plots
figure
subplot(3,1,1)
stem(n,h_hb)
xlabel('Samples')
ylabel('h_b[n]')
title('Impulse Response of H_b(z)')

subplot(3,1,2)
plot(w,abs(H_hb))
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
title('Magnitude Response of H_b(z)')

subplot(3,1,3)
pzplot(b_hb,a_hb)
axis equal
```

```
% % snapnow;
% ----------------------------------------------------------------
% 2(c)
% ----------------------------------------------------------------
```


Impulse Response of $H_a(z)$
$h_a[n]$
Samples
Magnitude Response of $H_a(z)$
Magnitude
Normalized Frequency (Radians)
Pole-Zero Plot
Imaginary Part
Real Part



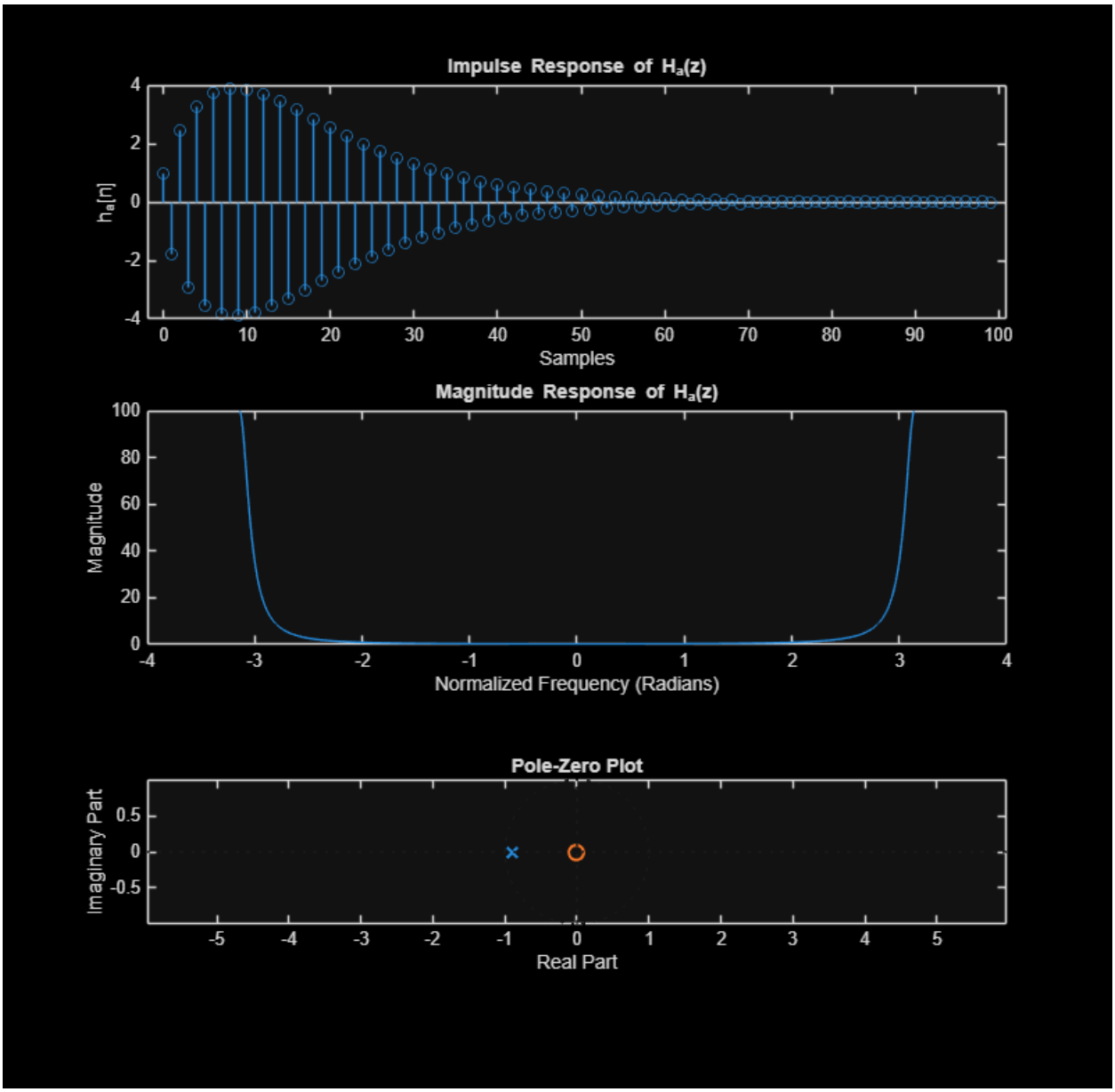
QUESTION 2(c)

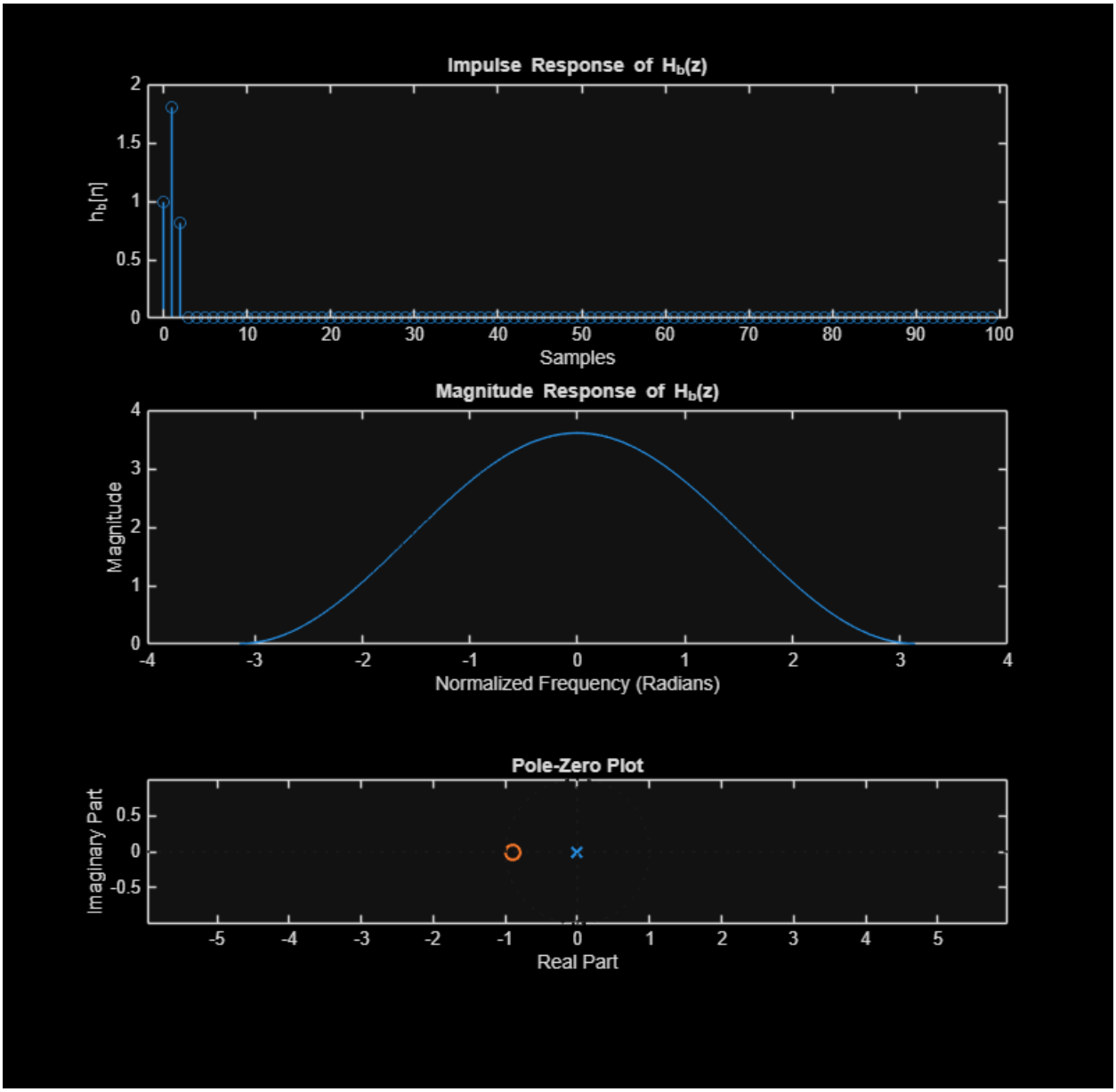
```
wa = pi;
wb = pi;
alpha = 0.9;
beta = 0.9;

N = 100;
n = 0:N-1;
x = zeros(N,1);
x(1) = 1;
w = -pi:pi/8000:pi-pi/8000;

% Ha(z)
```

```
a_ha = conv([1 -alpha*exp(1j*wa)], [1 -alpha*exp(-1j*wa)]);
b_ha = 1;

% Hb(z)
b_hb = conv([1 -beta*exp(1j*wb)], [1 -beta*exp(-1j*wb)]);
a_hb = 1;

% remove tiny imaginary roundoff
a_ha = real(a_ha);
b_hb = real(b_hb);

% impulse responses
h_ha = filter(b_ha, a_ha, x);
h_hb = filter(b_hb, a_hb, x);

% DTFTs
H_ha = DTFT(h_ha, w);
H_hb = DTFT(h_hb, w);

% Ha plots
figure
subplot(3,1,1)
stem(n,h_ha)
xlabel('Samples')
ylabel('h_a[n]')
title('Impulse Response of H_a(z)')

subplot(3,1,2)
plot(w,abs(H_ha))
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
title('Magnitude Response of H_a(z)')

subplot(3,1,3)
pzplot(b_ha,a_ha)
axis equal

% Hb plots
figure
subplot(3,1,1)
stem(n,h_hb)
xlabel('Samples')
ylabel('h_b[n]')
title('Impulse Response of H_b(z)')

subplot(3,1,2)
plot(w,abs(H_hb))
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
title('Magnitude Response of H_b(z)')

subplot(3,1,3)
pzplot(b_hb,a_hb)
axis equal
```

```
% % snapnow;
% ------------------------------------------------------------------
% 2(d)
% ------------------------------------------------------------------
% ANSWER QUESTION BELOW
%
% % Changing ωa and ωb shifts the location (angle) of the poles and zeros
% around the unit circle, which changes the frequency that is affected.
%
% In the pole-zero plots:
% - When ω = 0, the poles/zeros are on the positive real axis (Figures 1 &
2).
% - When ω = π/2, they move to the imaginary axis at ±j (Figures 3 & 4).
% - When ω = π, they move to the negative real axis at -1 (Figures 5 & 6).
%
% In the frequency responses:
% - For Ha(z) (poles), the magnitude peaks at the same ω location as the
poles.
%   For example, peak at 0 (Fig 1), peak at π/2 (Fig 3), peak at π (Fig 5).
% - For Hb(z) (zeros), the magnitude has a dip (notch) at the same ω
location.
%   For example, dip at 0 (Fig 2), dip at π/2 (Fig 4), dip at π (Fig 6).
%
% In the impulse responses:
% - For Ha(z), changing ωa changes the oscillation pattern of the decaying
IIR response.
%   At ω = 0 there is no oscillation (Figure 1), at ω = π/2 there is an
oscillating decaying response
%   (Figure 3), and at ω = π the response alternates sign every sample
(Figure 5).
% - For Hb(z), changing ωb changes the FIR coefficients and which samples
are nonzero.
%   For example, Figure 2 has three nonzero samples with a negative middle
term, Figure 4 has only
%   two visible nonzero samples because the middle coefficient is zero, and
Figure 6 has three
%   positive nonzero samples.



% ------------------------------------------------------------------
% 2(e)
% ------------------------------------------------------------------
% ANSWER QUESTION BELOW
%
% % Changing α and β changes the radius of the poles and zeros, which affects
% how strong and sharp the filter behavior is.
%
% In the pole-zero plots:
% - Increasing α or β moves the poles/zeros closer to the unit circle.
% - Decreasing α or β moves them closer to the origin.
%
% In the frequency responses:
```

```
% - For Ha(z), increasing α (poles closer to unit circle) makes the peak
%   sharper and larger. Decreasing α makes the peak smaller and wider.
% - For Hb(z), increasing β (zeros closer to unit circle) makes the notch
%   deeper and sharper. Decreasing β makes the notch less pronounced.
%
% In the impulse responses:
% - For Ha(z), increasing α causes the response to decay more slowly,
%   while decreasing α makes it decay faster.
% - For Hb(z), changing β only changes the values of the few FIR coefficients
%   (amplitudes of the impulses), not the length of the response.
%
```





==========

==========

QUESTION 3: IIR FILTERS IN Z-DOMAIN

==========

==========

```
% BUILD INPUT SIGNAL
N = 1000; n = 0:N-1; fs = 8000;
```

```
w = -pi:pi/500:pi-pi/500;
x = [cos( (5*pi/8)*n )  zeros(1,N)  sin( (2*pi/8)*n )  zeros(1,N)
3*cos( (6*pi/8)*n + 3*pi/4)  zeros(1,N)   ...
     cos( (3*pi/8)*n )  zeros(1,N)   2*cos( (1*pi/8)*n - 2*pi/4)
zeros(1,N)   0.5*cos( (4*pi/8)*n - pi/8)];

% ------------------------------------------------------------------
% 3(a)
% ------------------------------------------------------------------

X = DTFT(x,w);

figure
subplot(2,1,1)
plot(0:length(x)-1, x)
xlabel('Time (Samples)')
ylabel('x[n]')
title('Time-Domain Input Signal')

subplot(2,1,2)
plot(w, abs(X))
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
title('Magnitude Response of Input Signal')


%Plotted the same thing in a new figure belwo with time in seconds as the
%axis since I was unsure which we were supposed to do.

% time axis in seconds
t = (0:length(x)-1)/fs;


% DTFT
X = DTFT(x,w);

figure
subplot(2,1,1)
plot(t, x)
xlabel('Time (Seconds)')
ylabel('x[n]')
title('Time-Domain Input Signal')

subplot(2,1,2)
plot(w, abs(X))
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
title('Magnitude Response of Input Signal')

% % snapnow;
% ------------------------------------------------------------------
% 3(b)
% ------------------------------------------------------------------
```

```
% impulse input
N = 200;
n = 0:N-1;
delta = zeros(N,1);
delta(1) = 1;

% ---- Filter 1: Hb (5π/8)
beta = 1/0.999;
w0 = 5*pi/8;
b1 = conv([1 -beta*exp(1j*w0)], [1 -beta*exp(-1j*w0)]);
a1 = 1;

% ---- Filter 2: Ha (5π/8)
alpha = 0.999;
a2 = conv([1 -alpha*exp(1j*w0)], [1 -alpha*exp(-1j*w0)]);
b2 = 1;

% ---- Filter 3: Hb (6π/8)
beta = 1/0.999;
w0 = 6*pi/8;
b3 = conv([1 -beta*exp(1j*w0)], [1 -beta*exp(-1j*w0)]);
a3 = 1;

% ---- Filter 4: Ha (6π/8)
alpha = 0.999;
a4 = conv([1 -alpha*exp(1j*w0)], [1 -alpha*exp(-1j*w0)]);
b4 = 1;

% remove tiny imaginary parts
b1 = real(b1); a2 = real(a2);
b3 = real(b3); a4 = real(a4);

% cascade filtering (impulse response)
h = filter(b4,a4, ...
    filter(b3,a3, ...
    filter(b2,a2, ...
    filter(b1,a1, delta))));

% DTFT
H = DTFT(h,w);

% plots
figure
subplot(2,1,1)
stem(n,h)
xlabel('Samples')
ylabel('h[n]')
title('Impulse Response of Filter Cascade 1')

subplot(2,1,2)
plot(w, abs(H))
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
title('Magnitude Response of Filter Cascade 1')
```

```
% % snapnow;
% ------------------------------------------------------------------
% 3(c)
% ------------------------------------------------------------------


% apply cascade 1 to input x
y1 = filter(b4,a4, ...
     filter(b3,a3, ...
     filter(b2,a2, ...
     filter(b1,a1,x))));

% time axis in seconds
t = (0:length(y1)-1)/fs;

% DTFT of output
Y1 = DTFT(y1,w);

figure
subplot(2,1,1)
plot(t, y1)
xlabel('Time (Seconds)')
ylabel('y_1[n]')
title('Time-Domain Output of Filter Cascade 1')

subplot(2,1,2)
plot(w, abs(Y1))
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
title('Magnitude Response of Output for Filter Cascade 1')

% % snapnow;
% ------------------------------------------------------------------
% 3(d)
% ------------------------------------------------------------------

% impulse input
N = 200;
n = 0:N-1;
delta = zeros(N,1);
delta(1) = 1;

% ---- Ha (π/2)
alpha = 0.8;
w0 = pi/2;
a1 = conv([1 -alpha*exp(1j*w0)], [1 -alpha*exp(-1j*w0)]);
b1 = 1;

% ---- Hb filters (β = 1)
beta = 1;

% ω = 0
w0 = 0;
```

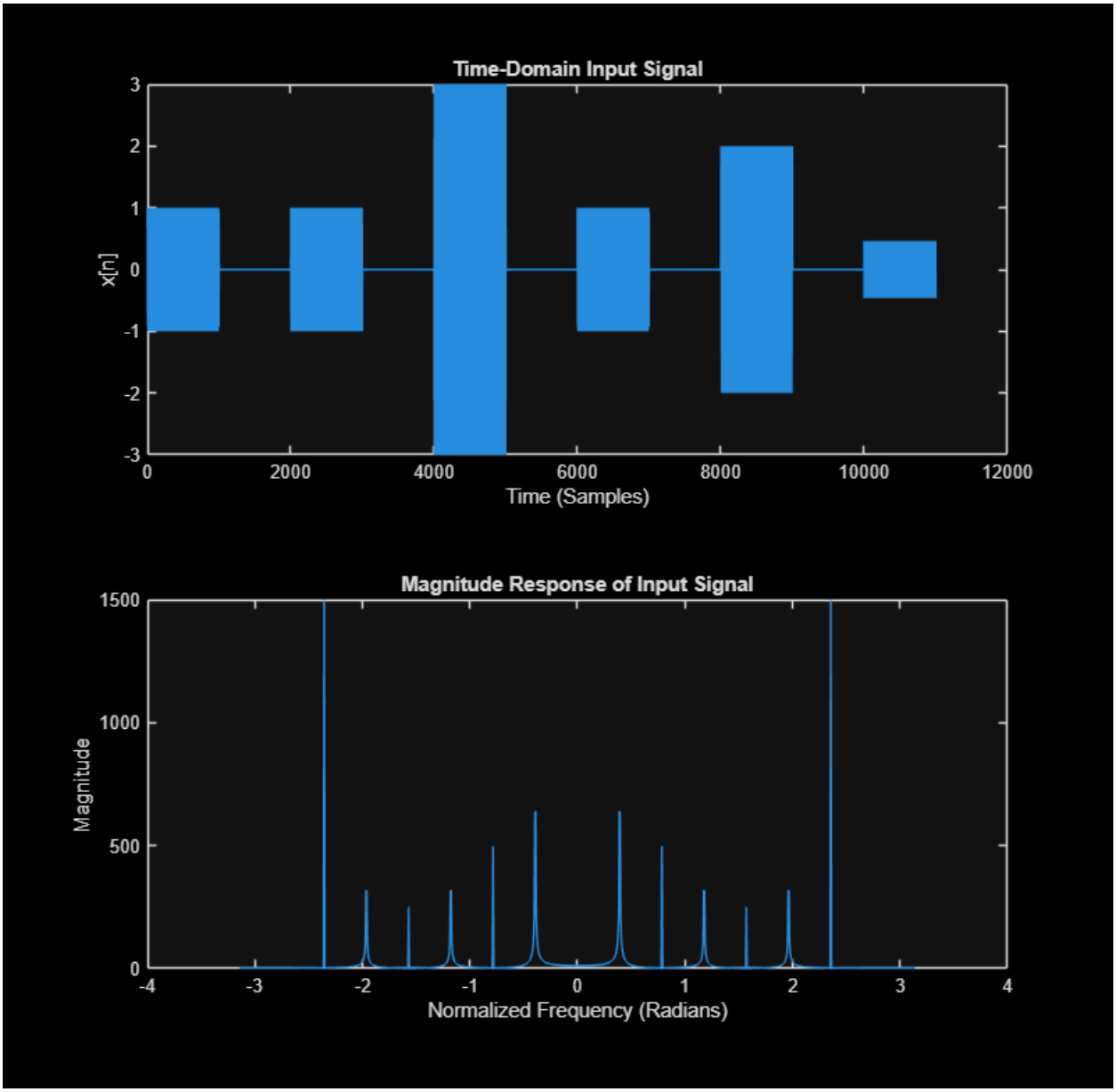
```
b2 = conv([1 -beta*exp(1j*w0)], [1 -beta*exp(-1j*w0)]);
a2 = 1;

% ω = π/4
w0 = pi/4;
b3 = conv([1 -beta*exp(1j*w0)], [1 -beta*exp(-1j*w0)]);
a3 = 1;

% ω = 3π/8
w0 = 3*pi/8;
b4 = conv([1 -beta*exp(1j*w0)], [1 -beta*exp(-1j*w0)]);
a4 = 1;

% ω = 5π/8
w0 = 5*pi/8;
b5 = conv([1 -beta*exp(1j*w0)], [1 -beta*exp(-1j*w0)]);
a5 = 1;

% ω = 3π/4
w0 = 3*pi/4;
b6 = conv([1 -beta*exp(1j*w0)], [1 -beta*exp(-1j*w0)]);
a6 = 1;

% ω = π
w0 = pi;
b7 = conv([1 -beta*exp(1j*w0)], [1 -beta*exp(-1j*w0)]);
a7 = 1;

% remove small imaginary parts
a1 = real(a1);
b2 = real(b2); b3 = real(b3); b4 = real(b4);
b5 = real(b5); b6 = real(b6); b7 = real(b7);

% cascade filtering (impulse response)
h2 = filter(b7,a7, ...
     filter(b6,a6, ...
     filter(b5,a5, ...
     filter(b4,a4, ...
     filter(b3,a3, ...
     filter(b2,a2, ...
     filter(b1,a1, delta)))))));

% DTFT
H2 = DTFT(h2,w);

% plots
figure
subplot(2,1,1)
stem(n,h2)
xlabel('Samples')
ylabel('h[n]')
title('Impulse Response of Filter Cascade 2')

subplot(2,1,2)
```

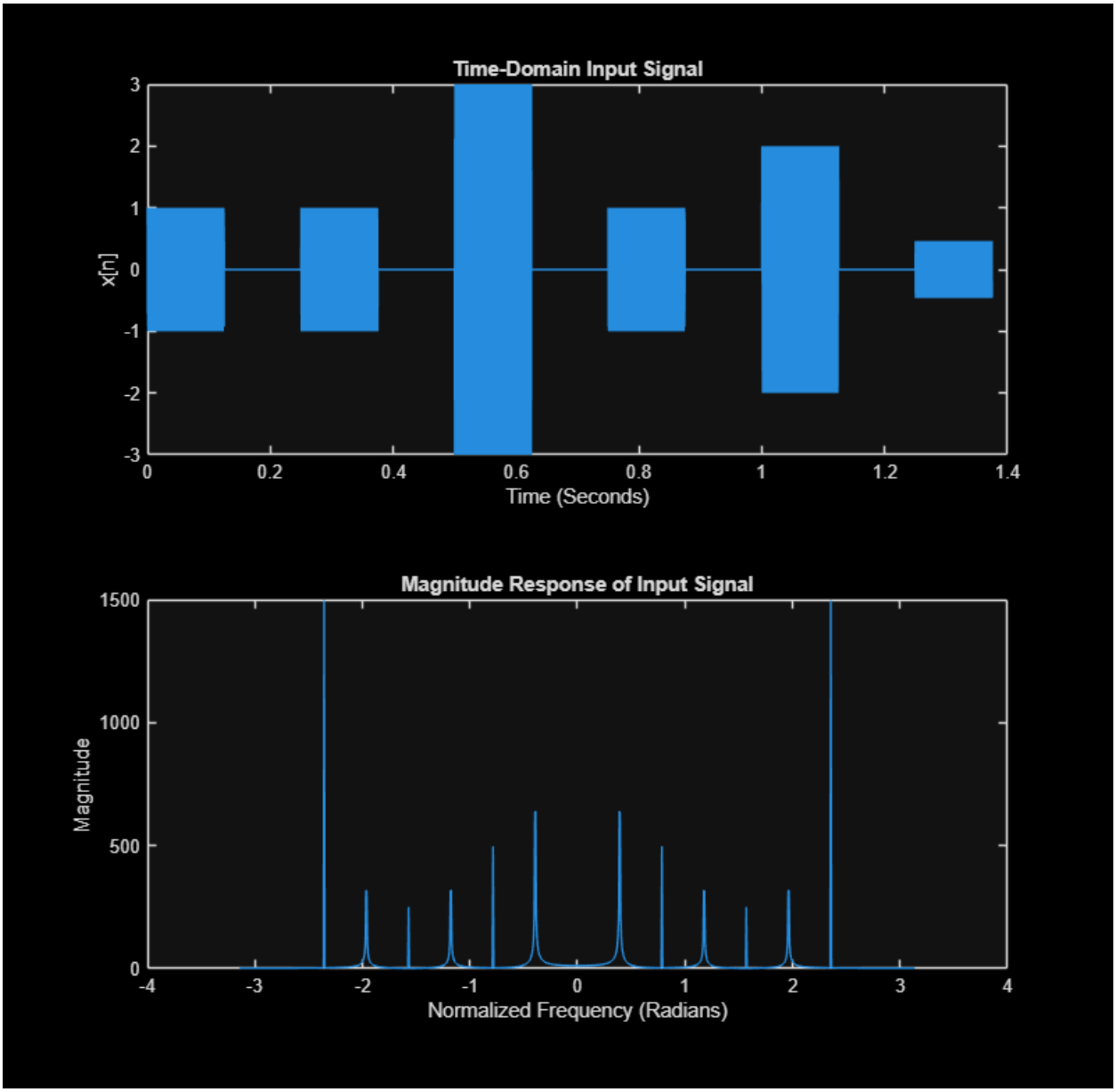
```
plot(w, abs(H2))
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
title('Magnitude Response of Filter Cascade 2')

% % snapnow;
% ----------------------------------------------------------------
% 3(e)
% ----------------------------------------------------------------

y2 = filter(b7,a7, ...
     filter(b6,a6, ...
     filter(b5,a5, ...
     filter(b4,a4, ...
     filter(b3,a3, ...
     filter(b2,a2, ...
     filter(b1,a1,x)))))));

t = (0:length(y2)-1)/fs;
Y2 = DTFT(y2,w);

figure
subplot(2,1,1)
plot(t, y2)
xlabel('Time (Seconds)')
ylabel('y_2[n]')
title('Time-Domain Output of Filter Cascade 2')

subplot(2,1,2)
plot(w, abs(Y2))
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
title('Magnitude Response of Output for Filter Cascade 2')

% % snapnow;
% ----------------------------------------------------------------
% 3(f)
% ----------------------------------------------------------------


% impulse input
N = 200;
n = 0:N-1;
delta = zeros(N,1);
delta(1) = 1;

% ---- Ha (π/2)
alpha = 0.66;
w0 = pi/2;
a1 = conv([1 -alpha*exp(1j*w0)], [1 -alpha*exp(-1j*w0)]);
b1 = 1;

% ---- Ha (π/8)
w0 = pi/8;
```

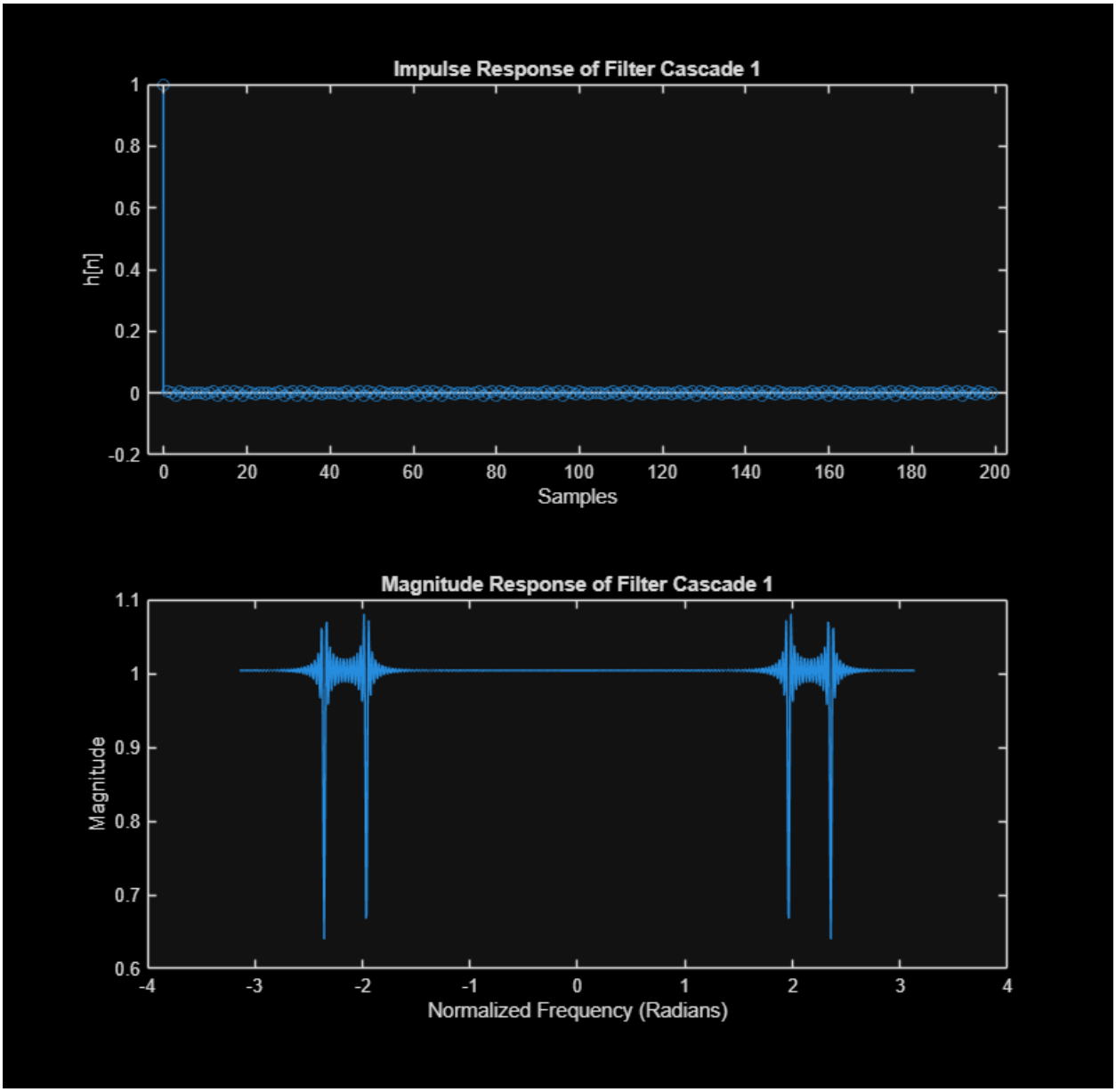
```
a2 = conv([1 -alpha*exp(1j*w0)], [1 -alpha*exp(-1j*w0)]);
b2 = 1;

% ---- Hb (π)
beta = 1;
w0 = pi;
b3 = conv([1 -beta*exp(1j*w0)], [1 -beta*exp(-1j*w0)]);
a3 = 1;

% remove tiny imaginary parts
a1 = real(a1); a2 = real(a2);
b3 = real(b3);

% cascade filtering (impulse response)
h3 = filter(b3,a3, ...
     filter(b2,a2, ...
     filter(b1,a1, delta)));

% DTFT
H3 = DTFT(h3,w);

% plots
figure
subplot(2,1,1)
stem(n,h3)
xlabel('Samples')
ylabel('h[n]')
title('Impulse Response of Filter Cascade 3')

subplot(2,1,2)
plot(w, abs(H3))
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
title('Magnitude Response of Filter Cascade 3')

% % snapnow;

% ------------------------------------------------------------------
% 3(g)
% ------------------------------------------------------------------

y3 = filter(b3,a3, ...
     filter(b2,a2, ...
     filter(b1,a1,x)));

t = (0:length(y3)-1)/fs;
Y3 = DTFT(y3,w);

figure
subplot(2,1,1)
plot(t, y3)
xlabel('Time (Seconds)')
ylabel('y_3[n]')
title('Time-Domain Output of Filter Cascade 3')
```

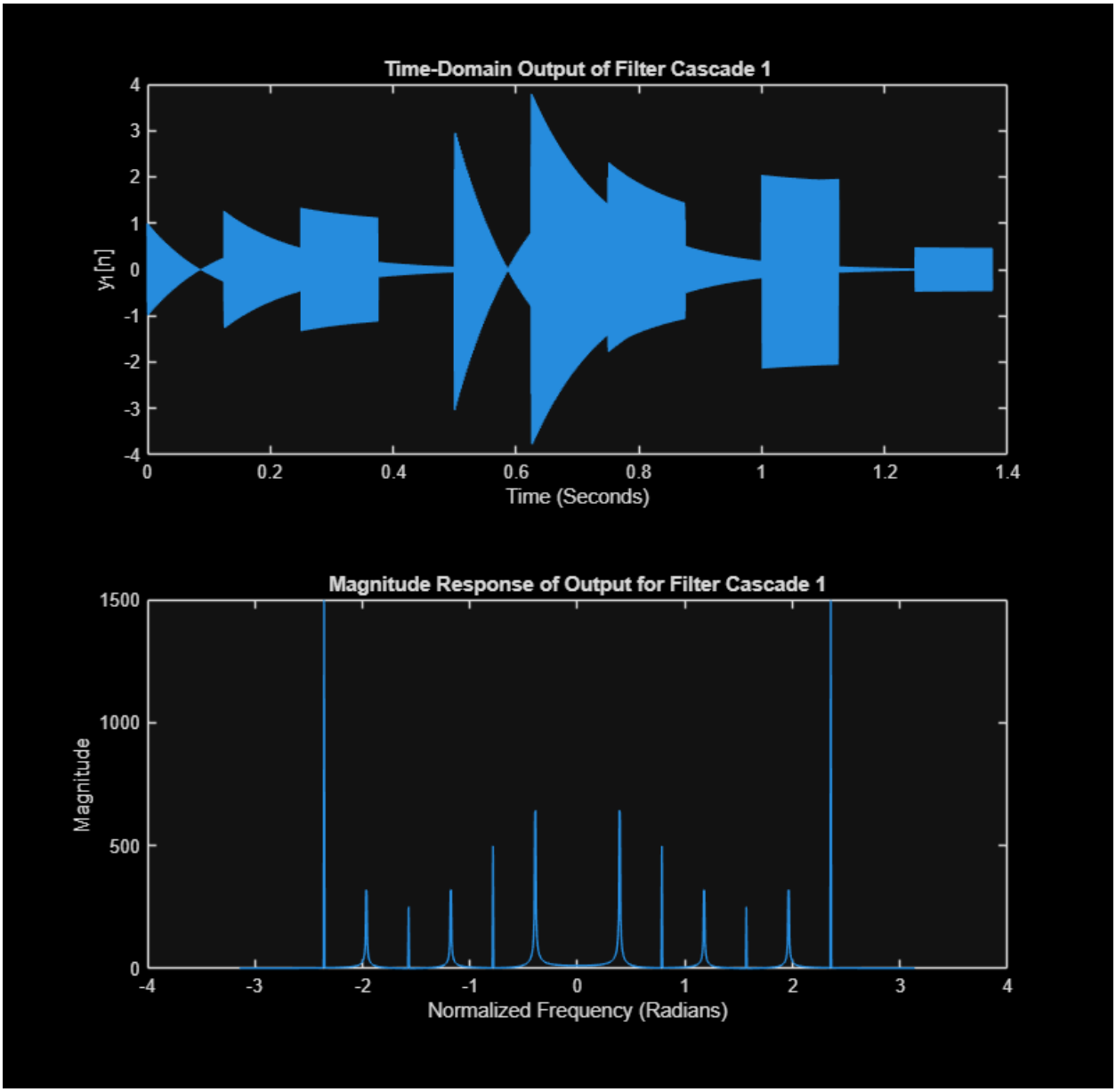
```
subplot(2,1,2)
plot(w, abs(Y3))
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
title('Magnitude Response of Output for Filter Cascade 3')

% % snapnow;

% ------------------------------------------------------------------
% 3(h)
% ------------------------------------------------------------------

% impulse input
N = 200;
n = 0:N-1;
delta = zeros(N,1);
delta(1) = 1;

% ---- Ha (π/3)
alpha = 0.9;
w0 = pi/3;
a1 = conv([1 -alpha*exp(1j*w0)], [1 -alpha*exp(-1j*w0)]);
b1 = 1;

% ---- Hb (π/3)
beta = 1;
b2 = conv([1 -beta*exp(1j*w0)], [1 -beta*exp(-1j*w0)]);
a2 = 1;

% remove tiny imaginary parts
a1 = real(a1);
b2 = real(b2);

% cascade filtering (impulse response)
h4 = filter(b2,a2, filter(b1,a1, delta));

% DTFT
H4 = DTFT(h4,w);

% plots
figure
subplot(2,1,1)
stem(n,h4)
xlabel('Samples')
ylabel('h[n]')
title('Impulse Response of Filter Cascade 4')

subplot(2,1,2)
plot(w, abs(H4))
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
title('Magnitude Response of Filter Cascade 4')
```

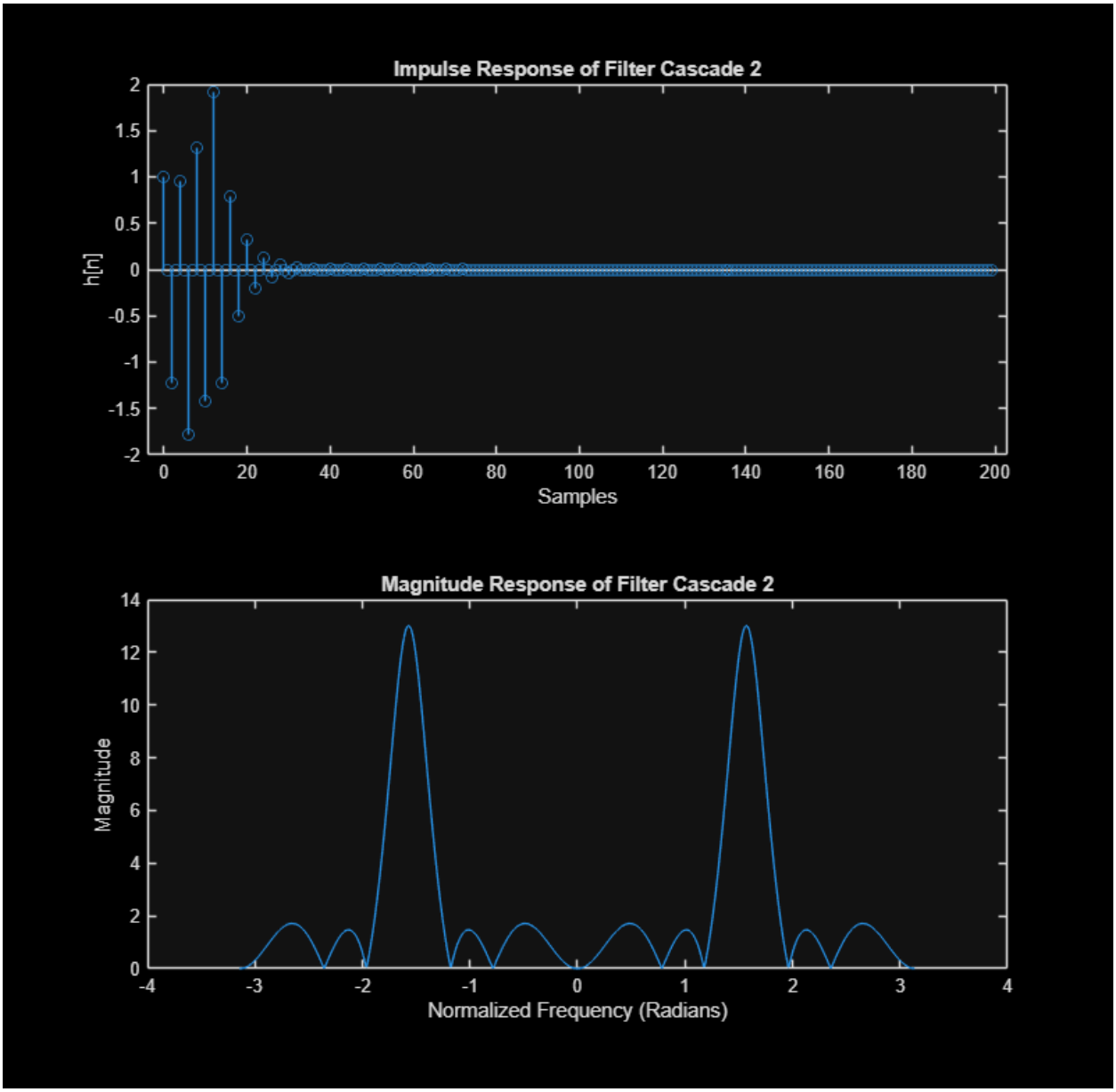
```
% % snapnow;

% ------------------------------------------------------------------
% 3(i)
% ------------------------------------------------------------------
y4 = filter(b2,a2, filter(b1,a1,x));

t = (0:length(y4)-1)/fs;
Y4 = DTFT(y4,w);

figure
subplot(2,1,1)
plot(t, y4)
xlabel('Time (Seconds)')
ylabel('y_4[n]')
title('Time-Domain Output of Filter Cascade 4')

subplot(2,1,2)
plot(w, abs(Y4))
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
title('Magnitude Response of Output for Filter Cascade 4')

% % snapnow;

% ------------------------------------------------------------------
% 3(j)
% ------------------------------------------------------------------
% ANSWER QUESTION BELOW
%
% Filter Cascade 1: Band-stop filter.
%
% Filter Cascade 2: Band-pass filter.
%
% Filter Cascade 3: Low-pass filter.
%
% Filter Cascade 4: Band-stop filter.
```

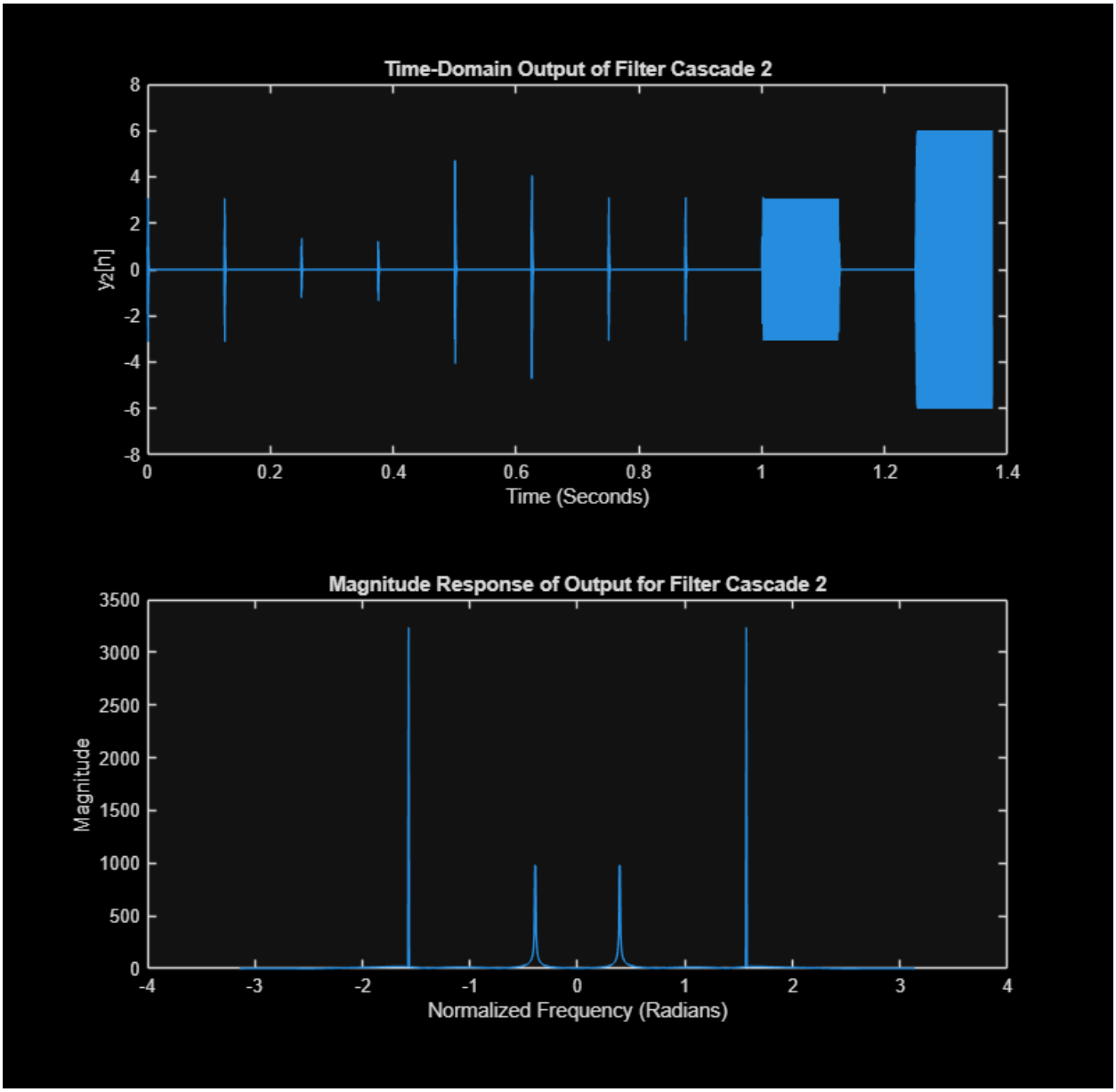

Time-Domain Input Signal
x[n]
Time (Samples)
Magnitude Response of Input Signal
Magnitude
Normalized Frequency (Radians)

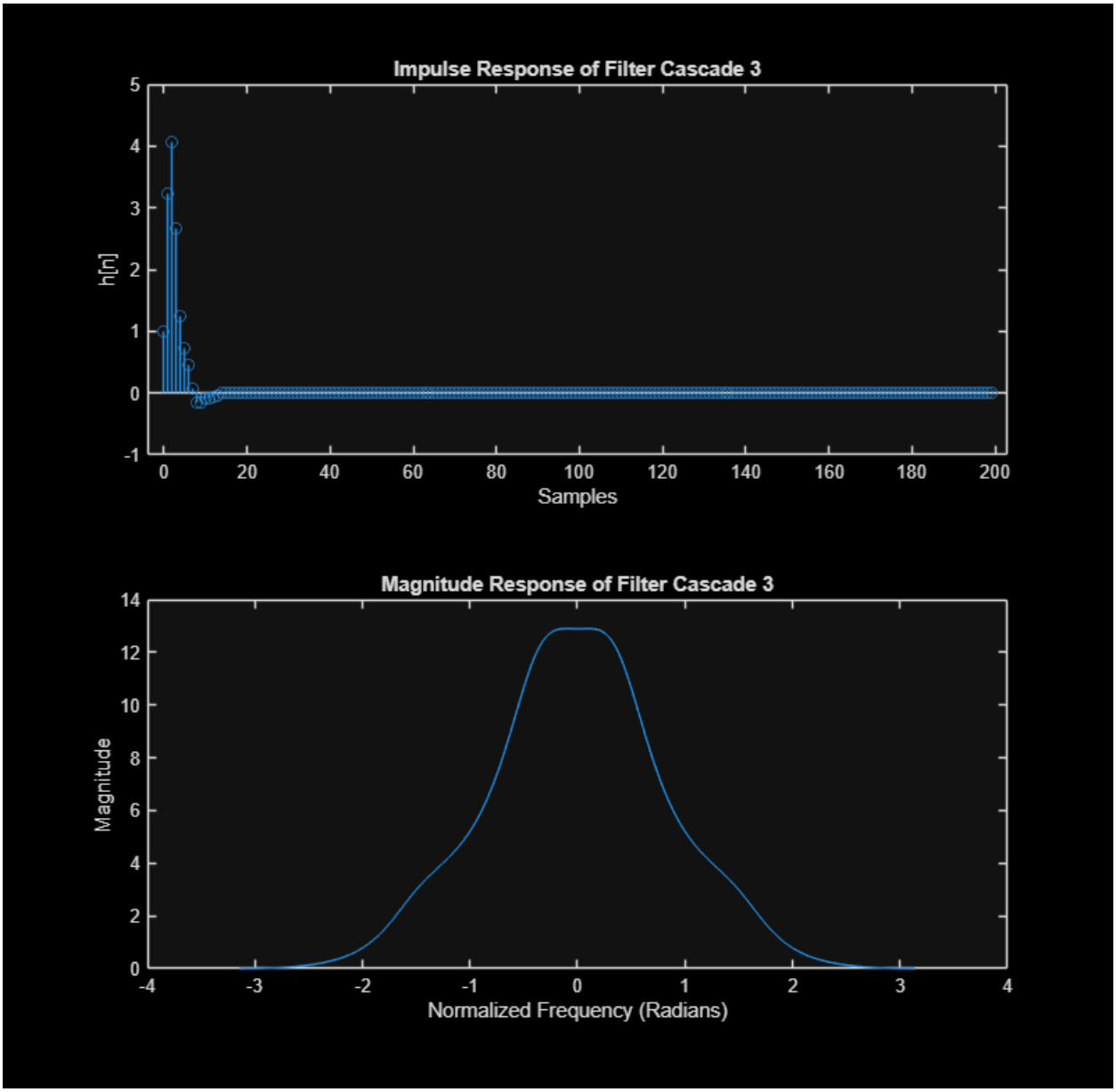

Time-Domain Input Signal
x[n]
Time (Seconds)
Magnitude Response of Input Signal
Magnitude
Normalized Frequency (Radians)

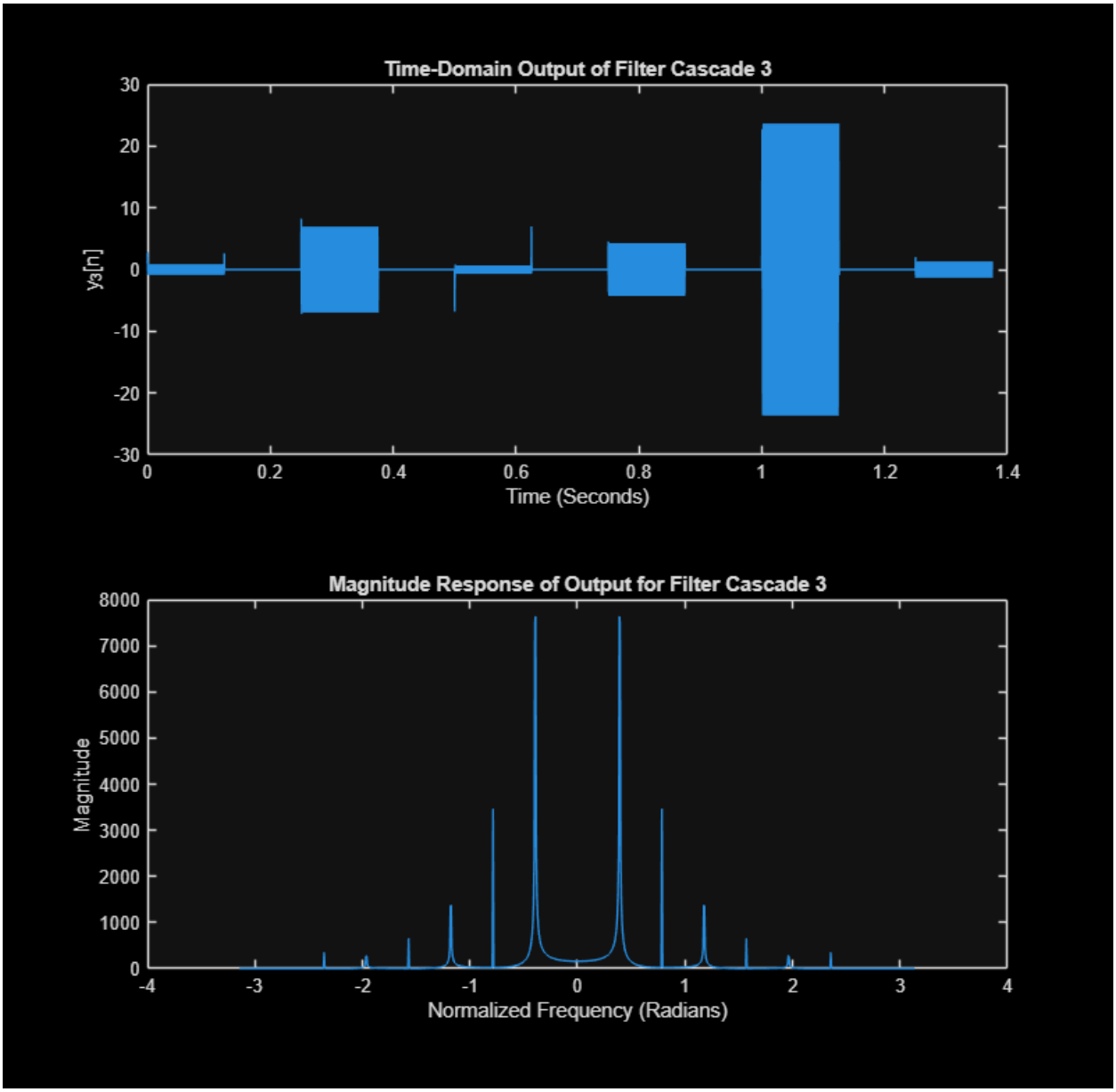

Impulse Response of Filter Cascade 1
h[n]
Samples
Magnitude Response of Filter Cascade 1
Magnitude
Normalized Frequency (Radians)

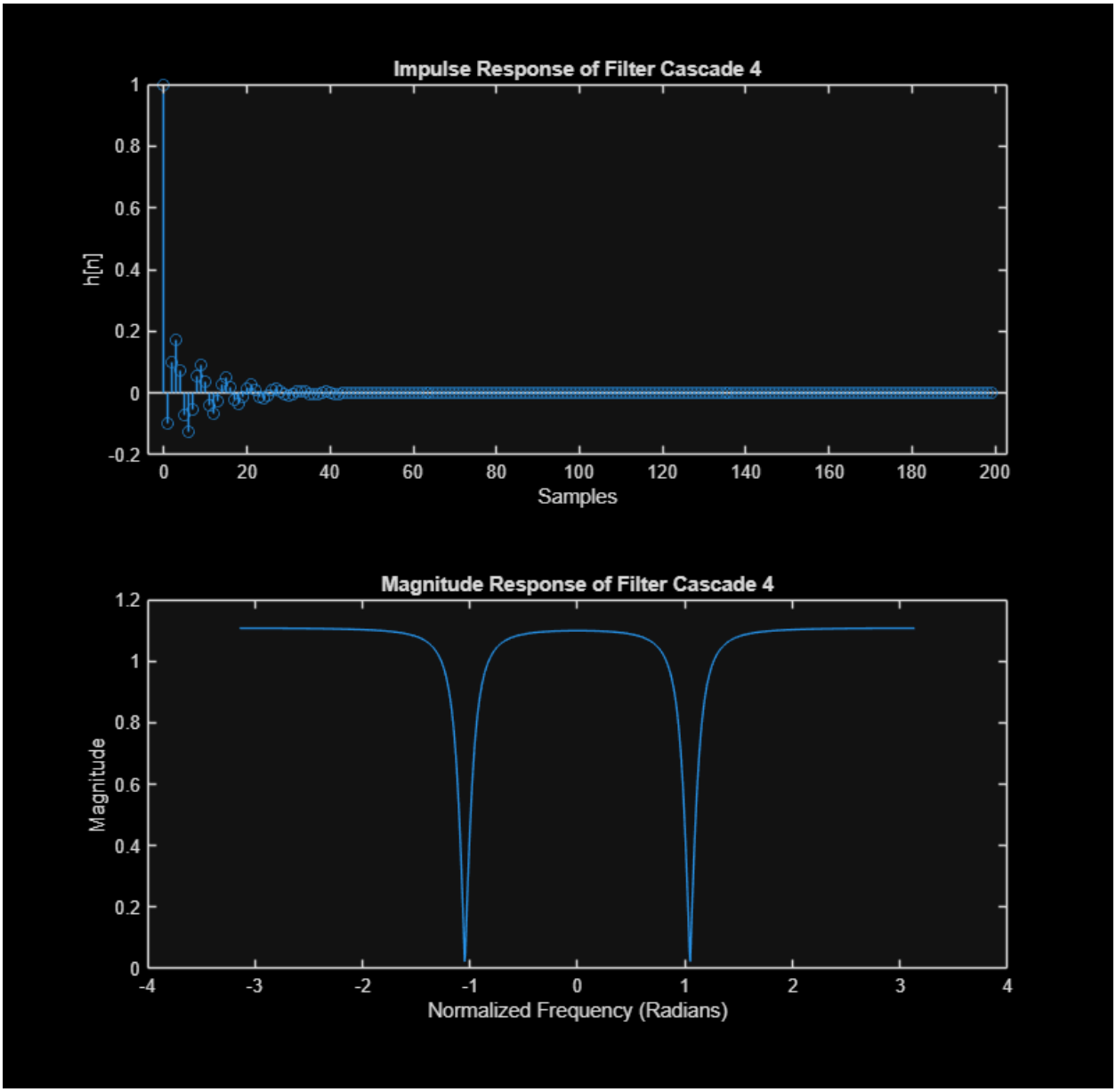

Time-Domain Output of Filter Cascade 1
y1[n]
Time (Seconds)
Magnitude Response of Output for Filter Cascade 1
Magnitude
Normalized Frequency (Radians)

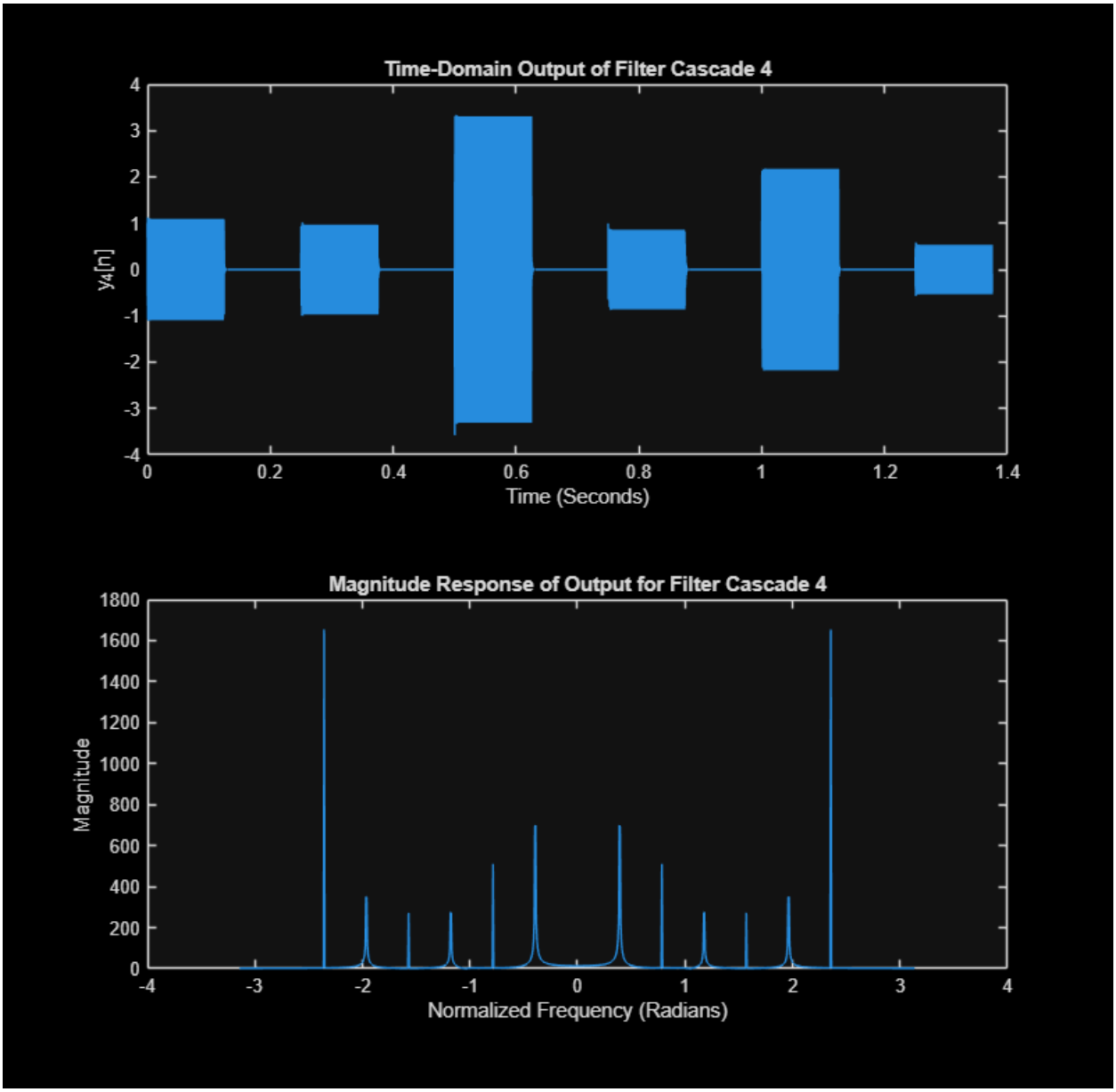

Impulse Response of Filter Cascade 2
h[n]
Samples
Magnitude Response of Filter Cascade 2
Magnitude
Normalized Frequency (Radians)


Time-Domain Output of Filter Cascade 2
y2[n]
Time (Seconds)
Magnitude Response of Output for Filter Cascade 2
Magnitude
Normalized Frequency (Radians)


Impulse Response of Filter Cascade 3
h[n]
Samples
Magnitude Response of Filter Cascade 3
Magnitude
Normalized Frequency (Radians)


Time-Domain Output of Filter Cascade 3
y3[n]
Time (Seconds)
Magnitude Response of Output for Filter Cascade 3
Magnitude
Normalized Frequency (Radians)


Impulse Response of Filter Cascade 4
h[n]
Samples
Magnitude Response of Filter Cascade 4
Magnitude
Normalized Frequency (Radians)



==========

==========

4. IIR Filter for Frequency Removal

==========

==========

```
% MAKE SURE FILES BELOW ARE IN SAME DIRECTORY AS THIS FILE
[x2, fs] = audioread('lab8_noisy2.wav');      % Audio for last part of problem
```

```
% ------------------------------------------------------------
% 4(a)
% ------------------------------------------------------------
% A better filter design strategy is to cascade multiple Hb(z) notch
% filters, placing zeros exactly on the unit circle at each identified
% noise frequency. The three noise frequencies in lab8_noisy2.wav were
% identified from the magnitude spectrum as approximately 0.377, 0.754,
% and 1.508 rad/sample. For each frequency w0, an Hb(z) section with
% zeros at ±w0 is cascaded together, allowing all three tonal noise
% components to be removed simultaneously in a single filter pass.
% This improves on Lab 6 by using the Z-domain cascade framework
% (H(z) = H1(z)*H2(z)*...) with the filter function rather than
% convolution, which is more numerically efficient and directly
% generalizable to IIR designs. Each notch targets only its specific
% frequency, preserving the surrounding audio content.



% ------------------------------------------------------------
% 4(b)
% ------------------------------------------------------------

x = x2(:,1);   % use one channel if stereo

% noise frequencies from Lab 6
w_noise = [0.376991; 0.753982; 1.50796];

% start with H(z) = 1
b_total = 1;
a_total = 1;

% cascade Hb(z) sections only (zeros only)
for k = 1:length(w_noise)
    w0 = w_noise(k);
    b_k = conv([1 -exp(1j*w0)], [1 -exp(-1j*w0)]);
    b_total = conv(b_total, b_k);
end

% remove tiny imaginary roundoff
b_total = real(b_total);

% filter signal
y = filter(b_total, a_total, x);

% normalize before saving
y = y / max(abs(y));

% save file
audiowrite('lab8_denoised2.wav', y, fs);
disp('Saved filtered audio as lab8_denoised2.wav')
```

```
% % snapnow;
```

Saved filtered audio as lab8_denoised2.wav

==========

==========

5. AI FOR REVERB FILTERING (IIR FILTERS)

==========

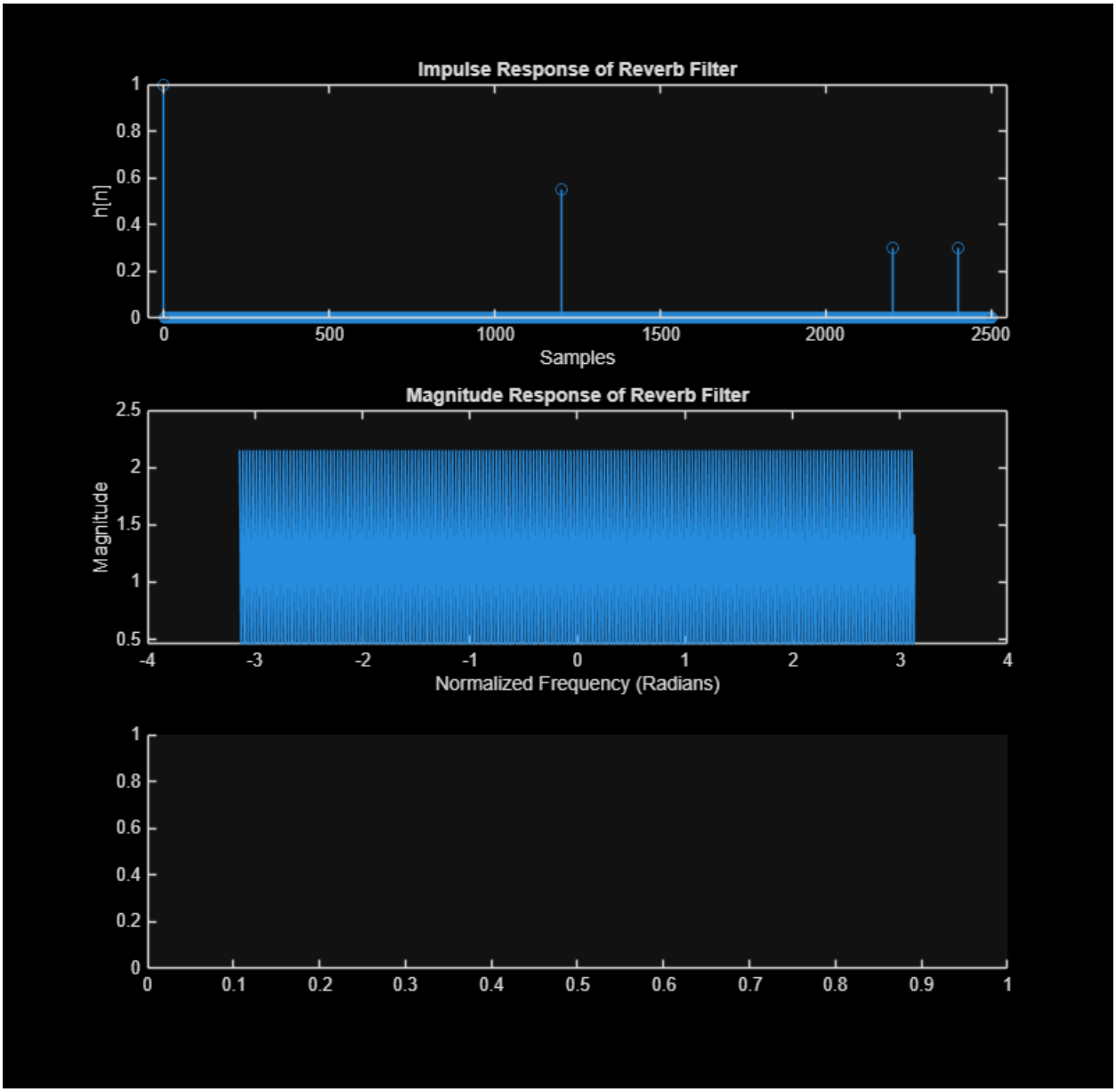
==========

---------- 5(a) ----------

PLACED FUNCTION AT END OF FILE

```
% ------------------------------------------------------------
% 5(b)
% ------------------------------------------------------------
% load drum solo
[x_drum, fs_drum] = audioread('drum_solo.wav');
x_drum = x_drum(:,1);

% get filter coefficients
[y_reverb, b_reverb, a_reverb] = reverb_filter(x_drum);

% impulse response
N = 2500;
n = 0:N-1;
delta = zeros(1,N);
delta(1) = 1;
h_reverb = filter(b_reverb, a_reverb, delta);

% frequency response
w = -pi:pi/500:pi-pi/500;
H_reverb = DTFT(h_reverb, w);

% plots
figure
subplot(3,1,1)
stem(n, h_reverb)
xlabel('Samples')
ylabel('h[n]')
title('Impulse Response of Reverb Filter')

subplot(3,1,2)
plot(w, abs(H_reverb))
xlabel('Normalized Frequency (Radians)')
ylabel('Magnitude')
title('Magnitude Response of Reverb Filter')
```

```
subplot(3,1,3)
pzplot(b_reverb, a_reverb)
axis equal

snapnow;

% ------------------------------------------------------------------
% 5(c)
% ------------------------------------------------------------------
% ANSWER QUESTION BELOW
% % The impulse response demonstrates a reverb effect because it contains
% multiple delayed and scaled copies of the original impulse.
%
% Instead of only having a single spike at n = 0, additional impulses appear
% at later sample indices (n = 1200, n = 2200, etc.), each with a
% smaller amplitude than the previous one.
%
% This shows that the system produces echoes of the input signal that are
% both delayed in time and decaying in magnitude.
%
% This behavior mimics real-world reverberation, where sound reflects off
% surfaces and arrives at the listener as multiple weaker, delayed copies,
% creating the perception of a reverberant space.
%



% ------------------------------------------------------------------
% 5(d)
% ------------------------------------------------------------------
% read input audio
[x, fs] = audioread('drum_solo.wav');

% use one channel if stereo
x = x(:,1);

% apply your reverb filter
[y, b, a] = reverb_filter(x);

% normalize to avoid clipping
y = y / max(abs(y));

% save output audio
audiowrite('reverb_ai.wav', y, fs);

disp('Saved filtered audio as reverb_ai.wav')
```



ALL FUNCTIONS SUPPORTING THIS CODE % % %

==========

==========

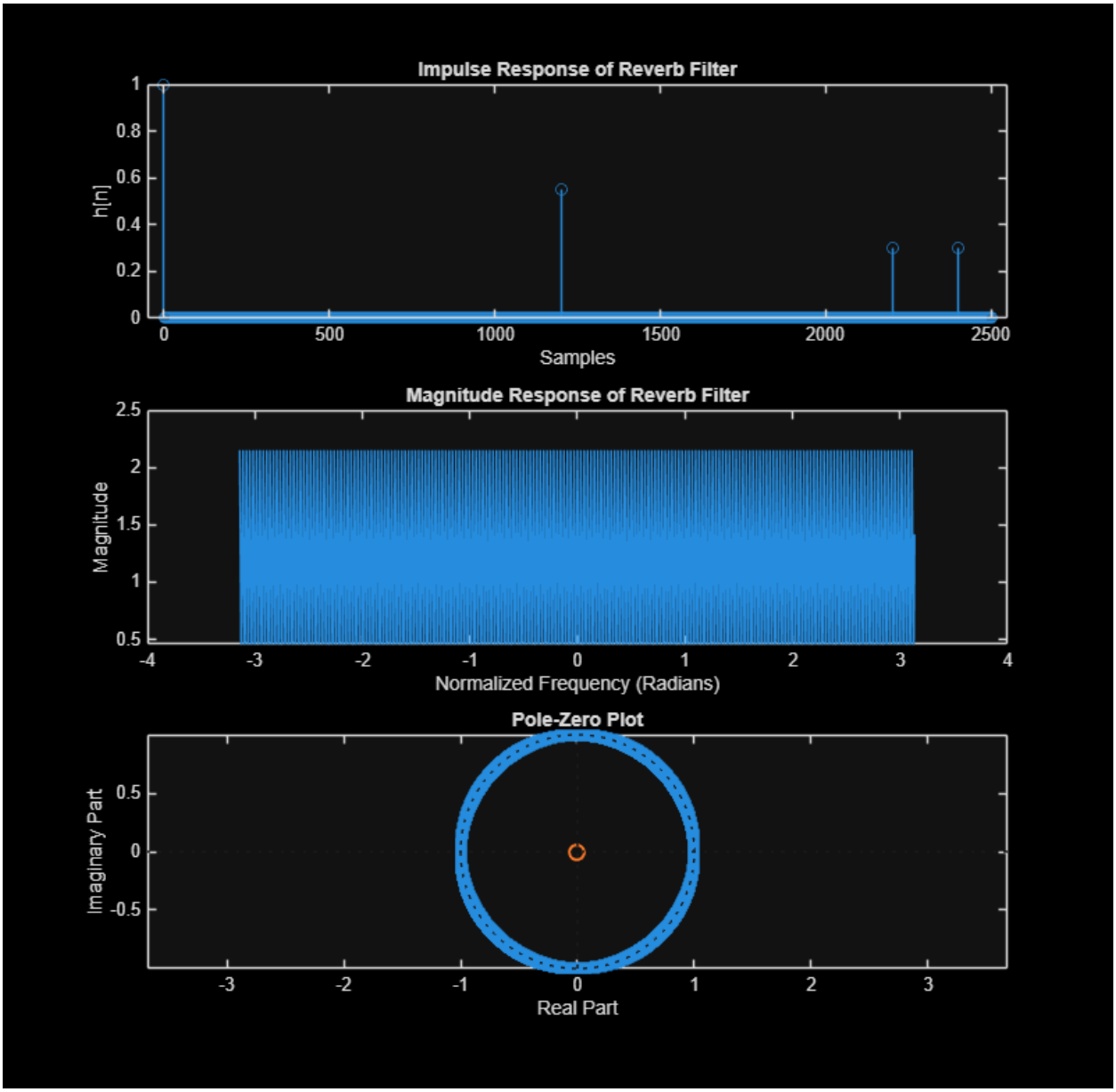
NOTE: YOU DO NOT NEED TO ADD COMMENTS IN THE CODE BELOW. WE JUST NEEDED POLE-ZERO PLOTTING CODE AND THUS WROTE IT.

==========

```
function pzplot(b,a)
% PZPLOT(B,A)  plots the pole-zero plot for the filter described by
```

```
% vectors A and B.  The filter is a "Direct Form II Transposed"
% implementation of the standard difference equation:
%
%    a(1)*y(n) = b(1)*x(n) + b(2)*x(n-1) + ... + b(nb+1)*x(n-nb)
%                          - a(2)*y(n-1) - ... - a(na+1)*y(n-na)
%

    % MODIFY THE POLYNOMIALS TO FIND THE ROOTS
    b1 = zeros(max(length(a),length(b)),1); % Need to add zeros to get the
right roots
    a1 = zeros(max(length(a),length(b)),1); % Need to add zeros to get the
right roots
    b1(1:length(b)) = b;    % New a with all values
    a1(1:length(a)) = a;    % New a with all values

    % FIND THE ROOTS OF EACH POLYNOMIAL AND PLOT THE LOCATIONS OF THE ROOTS
    h1 = plot(real(roots(a1)), imag(roots(a1)));
    hold on;
    h2 = plot(real(roots(b1)), imag(roots(b1)));
    hold off;

    % DRAW THE UNIT CIRCLE
    circle(0,0,1)

    % MAKE THE POLES AND ZEROS X's AND O's
    set(h1, 'LineStyle', 'none', 'Marker', 'x', 'MarkerFaceColor','none',
'linewidth', 1.5, 'markersize', 8);
    set(h2, 'LineStyle', 'none', 'Marker', 'o', 'MarkerFaceColor','none',
'linewidth', 1.5, 'markersize', 8);
    axis equal;

    % DRAW VERTICAL AND HORIZONTAL LINES
    xminmax = xlim();
    yminmax = ylim();
    line([xminmax(1) xminmax(2)],[0 0], 'linestyle', ':', 'linewidth', 0.5,
'color', [1 1 1]*.1)
    line([0 0],[yminmax(1) yminmax(2)], 'linestyle', ':', 'linewidth', 0.5,
'color', [1 1 1]*.1)

    % ADD LABELS AND TITLE
    xlabel('Real Part')
    ylabel('Imaginary Part')
    title('Pole-Zero Plot')

end

function circle(x,y,r)
% CIRCLE(X,Y,R)  draws a circle with horizontal center X, vertical center
% Y, and radius R.
%

    % ANGLES TO DRAW
    ang=0:0.01:2*pi;
```

```
    % DEFINE LOCATIONS OF CIRCLE
    xp=r*cos(ang);
    yp=r*sin(ang);

    % PLOT CIRCLE
    hold on;
    plot(x+xp,y+yp, ':', 'linewidth', 0.5, 'color', [1 1 1]*.1);
    hold off;


end

function H = DTFT(x,w)
% DTFT(X,W)  compute the Discrete-time Fourier Transform of signal X
% across frequencies defined by W.

    H = zeros(length(w),1);
    for nn = 1:length(x)
        H = H + x(nn).*exp(-1j*w.'*(nn-1));
    end

end




function [y, b, a] = reverb_filter(x)

    % perceptible delays
    D1 = 1200;
    D2 = 2200;

    % feedback gains (small enough for stability / decay)
    g1 = 0.55;
    g2 = 0.30;

    % numerator and denominator
    b = 1;
    a = zeros(1, D2 + 1);
    a(1) = 1;
    a(D1 + 1) = -g1;
    a(D2 + 1) = -g2;

    % apply filter
    y = filter(b, a, x);

    % normalize output
    y = y / max(abs(y));
end
```




```
Saved filtered audio as reverb_ai.wav
```

Published with MATLAB® R2025b